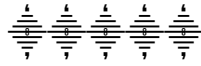


PHENIKAA UNIVERSITY
PHENIKAA SCHOOL OF COMPUTING



COURSE: SOFTWARE ARCHITECTURE

TOPIC:

QR CODE-BASED IN-TABLE FOOD ORDERING SYSTEM REPORT
LAB 4 – MICROSERVICES DECOMPOSITION AND COMMUNICATION

Code Course : CSE703110-1-3-25

Class : N01

Instructor : THS. Vũ Quang Dũng

Group : 01

Members : 1. Nguyễn Đình Trung - 23010701
2. Vũ Anh Nam - 23010886
3. Đặng Minh Hoàng – 23010445
4. Nguyễn Phan Quang Minh - 23010530

Table of Contents

1. Overview and Alignment with Lab 4.....	3
2. System Overview – UniMIS (Current MVP).....	3
3. Decomposition by Business Capability	3
4. External Dependencies	4
5. Service and Module Contracts.....	4
6. Communication Strategy (Synchronous vs. Asynchronous)	5
7. C4 Model – Level 1 (System Context).....	5
8. Conclusion.....	6

1. Overview and Alignment with Lab 4

Lab 4 focuses on the transition from a traditional monolithic architecture to a microservices-oriented design by emphasizing business capability decomposition, strict service boundaries, communication strategies, and system context modeling. The primary objective of this lab is not deployment implementation, but architectural reasoning and design clarity.

Although FOODASH is currently designed as a modular monolith, its architecture is intentionally engineered to follow microservices principles at the structural level. Each sub-domain is clearly decoupled by business capability, enforces absolute data ownership, and communicates with other domains through well-defined contracts. As a result, the FOODASH system fully aligns with the conceptual goals of Lab 4, even though it is packaged and deployed as a single application container.

2. System Overview – UniMIS (Current MVP)

FOODASH is a real-time, QR code-based tableside food ordering system designed to streamline menu browsing, order workflows, kitchen dispatch, and payment transactions within a restaurant context. The system provides real-time CRUD operations for digital menus, processes active multi-item cart submission, pushes instant assistance or cooking alerts to staff terminals, logs financial transactions, and generates centralized sales intelligence reports.

From an engineering implementation perspective, FOODASH consists of a high-concurrency Node.js and Express backend API app, a relational PostgreSQL primary database, an in-memory Redis cache store, and a React-based frontend Single-Page Application (SPA) built with modern JavaScript. Logically, FOODASH completely adopts a modular monolith architecture. Each business domain is implemented as an independent module with its own controller endpoints, service layer components, data schemas, and dedicated database models. While all modules execute within the same application runtime thread, they are tightly isolated to behave as if they were distributed microservices, closely mirroring a microservices architecture at the design level.

3. Decomposition by Business Capability

In accordance with Lab 4 requirements, FOODASH is decomposed strictly based on business capabilities rather than general technical layers. Each major business function of the digital restaurant domain is mapped to a dedicated module representing a potential

microservice boundary if the platform needs to scale out:

- **Menu & Catalog Management Module:** Owns the configurations for food/beverage items, catalog grouping layout configurations, base pricing schemes, and seasonal menu layout versions.
- **Table Tracking & QR Management Module:** Manages the physical layout mapping of seating infrastructure and generates unique, tokenized tableside QR code identifier keys.
- **Order & Assistance Real-Time Workflow Module:** Orchestrates the core transactional lifecycle of tableside orders (Pending $\rightarrow$ Cooking $\rightarrow$ Served $\rightarrow$ Completed) and pushes instant staff notification payloads.
- **Billing & Financial Calculation Module:** Governs multi-item bill subtotals, active promotional voucher deductions, tax calculations (VAT), and coordinates with the check-out flow.
- **User and Permission Management Module:** Handles system roles and administrative dashboard permissions for owners, supervisors, and kitchen staff.

A strict domain boundary rule is enforced across all internal modules. Even though all data tables reside inside a single physical PostgreSQL database container, no module is allowed to execute cross-boundary queries or directly read/write tables belonging to another module. When data from another domain is required, it must be requested via a public internal interface or a contract DTO exposed by the owning module. This design perfectly simulates the *"Database-per-Service"* principle of microservices and guarantees loose coupling..

4. External Dependencies

In addition to internal capability modules, FOODASH interfaces with critical external systems that lie outside the central core system boundary to complete operational workflows:

- **External Payment Gateway (e.g., VNPay):** Handles secure client-side banking transfers, digital wallet aggregations, and returns secure webhook transaction validation signals to confirm order payments.
- **Real-time Push Notification Gateway:** Interfaces with operational systems to route urgent assistance alerts or table order update pop-ups directly to mobile devices or smartwatch wear by waiting staff.

These external systems are isolated as independent architectural actors and are strictly accessed through well-defined API gateways or secure integration services.

5. Service and Module Contracts

Lab 4 emphasizes the vital role of service contracts as the main communication interfaces between system nodes. In FOODASH, this architecture is strictly maintained through public versioned REST APIs and explicit internal module interfaces. Public endpoints are hosted under a secure `/api/v1` namespace and are parsed by the browser frontend clients to handle user context, live menus, cart items, and payment links.

Internally, modules interact via rigorous producer-consumer contract schemas. A representative scenario occurs when a customer submits an active order string to the *Order & Assistance Real-Time Workflow Module* (Consumer). To validate the request, the Order module needs real-time price validation and stock check details belonging to the *Menu & Catalog Management Module* (Producer). Instead of executing an unauthorized direct database join on the menu tables, the Order module invokes a protected query method contract exposed by the Menu service interface to pull a read-only data snapshot. This interaction strictly prevents structural leaks and honors the decoupled boundaries defined in Lab 4

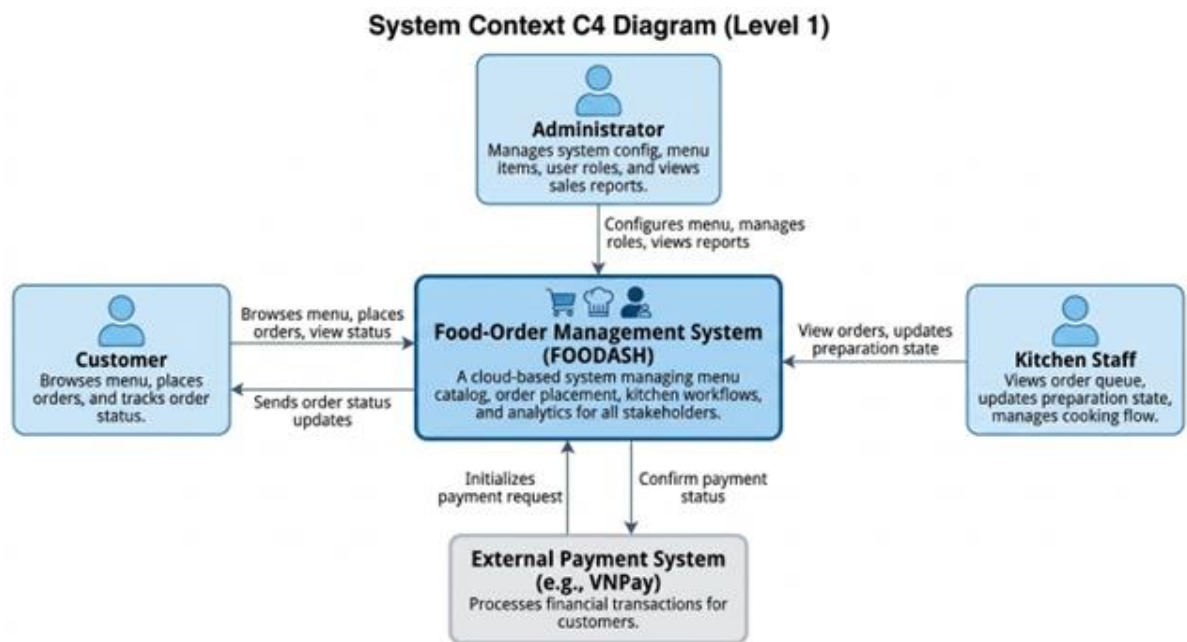
6. Communication Strategy (Synchronous vs. Asynchronous)

FOODASH coordinates data states using both synchronous and asynchronous communication patterns, precisely driven by business requirements and customer experience factors:

- **Synchronous Communication:** Deployed for operations demanding immediate, blocking client-side feedback. Examples include validating staff credentials, parsing tableside QR code checks, checking current cart totals, and initializing the secure external payment URL generation. These routines rely on fast HTTP request-response patterns over secure channels.

- **Asynchronous Communication:** Applied to long-running, heavy background workflows, or processes that must not block the frontend ordering journey. Examples include dispatching system-wide kitchen order alert tickets, broadcasting a tableside assistance call payload across multiple server terminals, updating secondary in-memory Redis cache items, and assembling daily financial sales report charts. These operations utilize active background workers or pub-sub handlers, guaranteeing high server responsiveness and fault-tolerant concurrency during peak restaurant dining hours.

7. C4 Model – Level 1 (System Context)



To satisfy the C4 Model Level 1 requirement, FOODASH is modeled as a unified system boundary interacting with localized users and remote external actors. The primary actors include Customers, Kitchen Staff, and Administrators. Within this system context framework, FOODASH functions as the central management engine through which customers place orders, kitchen staff manage preparation lines, and administrators maintain menus and audit sales reports. External boundaries, specifically the Payment Gateway (VNPay), interact securely with FOODASH to process financial checkouts. This high-level view establishes a complete functional understanding of how FOODASH integrates within the modern food-service operational ecosystem without exposing internal programming mechanics.

8. Conclusion

Although Lab 4 is traditionally framed around distributed microservices topologies, its architectural goals and abstraction design rules are fully realized within FOODASH through a strict modular monolithic design. By decomposing the application according to explicit business capabilities , enforcing zero-shared cross-module data boundaries , publishing service contracts , splitting real-time patterns into synchronous/asynchronous flows , and mapping actors via C4 Level 1 , FOODASH successfully satisfies all conceptual and analytical requirements of Lab 4. This clean layout provides an immediate, solid blueprint for future scale out into microservices if the platform encounters immense franchise growth.